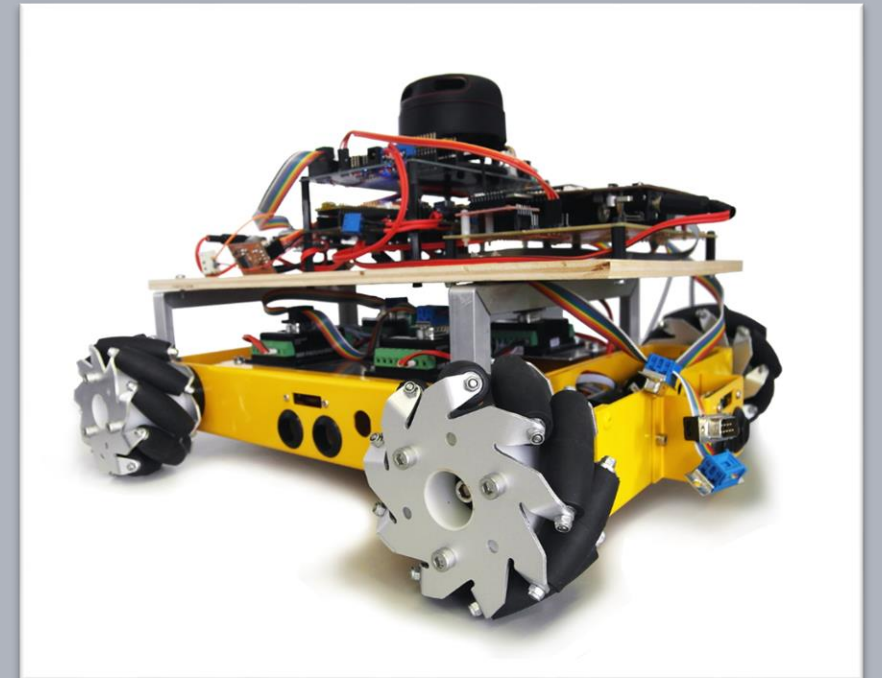


Advanced Embedded Architectures and Applications

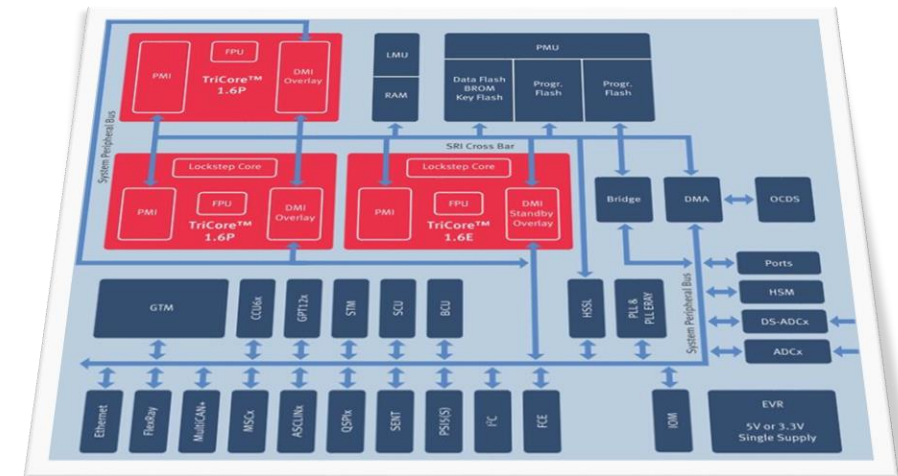
Multicore

Prof. Dr.-Ing. Peter Fromm



Content

- Trends in MCU Development
- Multicore Architectures
- The Infineon Aurix Platform
- Multicore Hello World
- PXROS
- Multicore RTE



Motivation for Multicore



Performance, physical design limitations



Combination of different domains,
e.g. real-time control and graphics in
one ECU



Increasing demand for functional
safety solutions

A multi-core processor is a single computing component with two or more independent actual central processing units (called "cores"), which are the units that read and execute program instructions. The instructions are ordinary CPU instructions such as add, move data, and branch, but the multiple cores can run multiple instructions at the same time, increasing overall speed for programs amenable to parallel computing. Manufacturers typically integrate the cores onto a single integrated circuit die (known as a chip multiprocessor or CMP), or onto multiple dies in a single chip package.

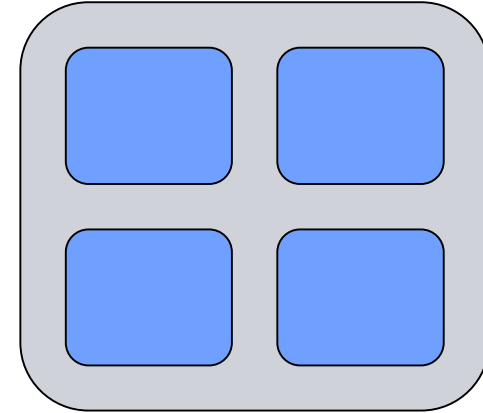
Year	Technology
1980	Rockwell: Introduction of 2 integrated 6502 CPU's on one die
2000	AMD and Intel: First multicore prototypes for the PC market
2010	First embedded multicore solutions (Renesas, TI, Infineon,...)

source: wikipedia

Hardware Architectures - 1

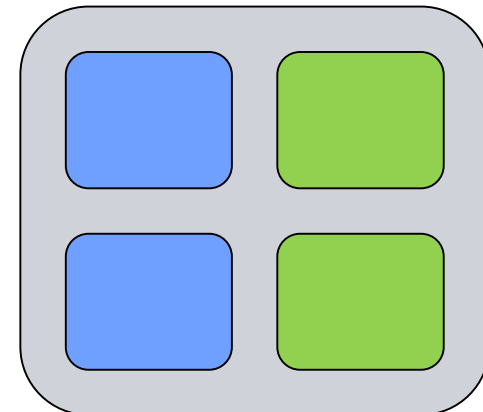
Homogeneous Multicore

- Provide multiple similar cores
- Goal: Performance increase, redundancy
- Example: Intel Core i7, Infineon Aurix

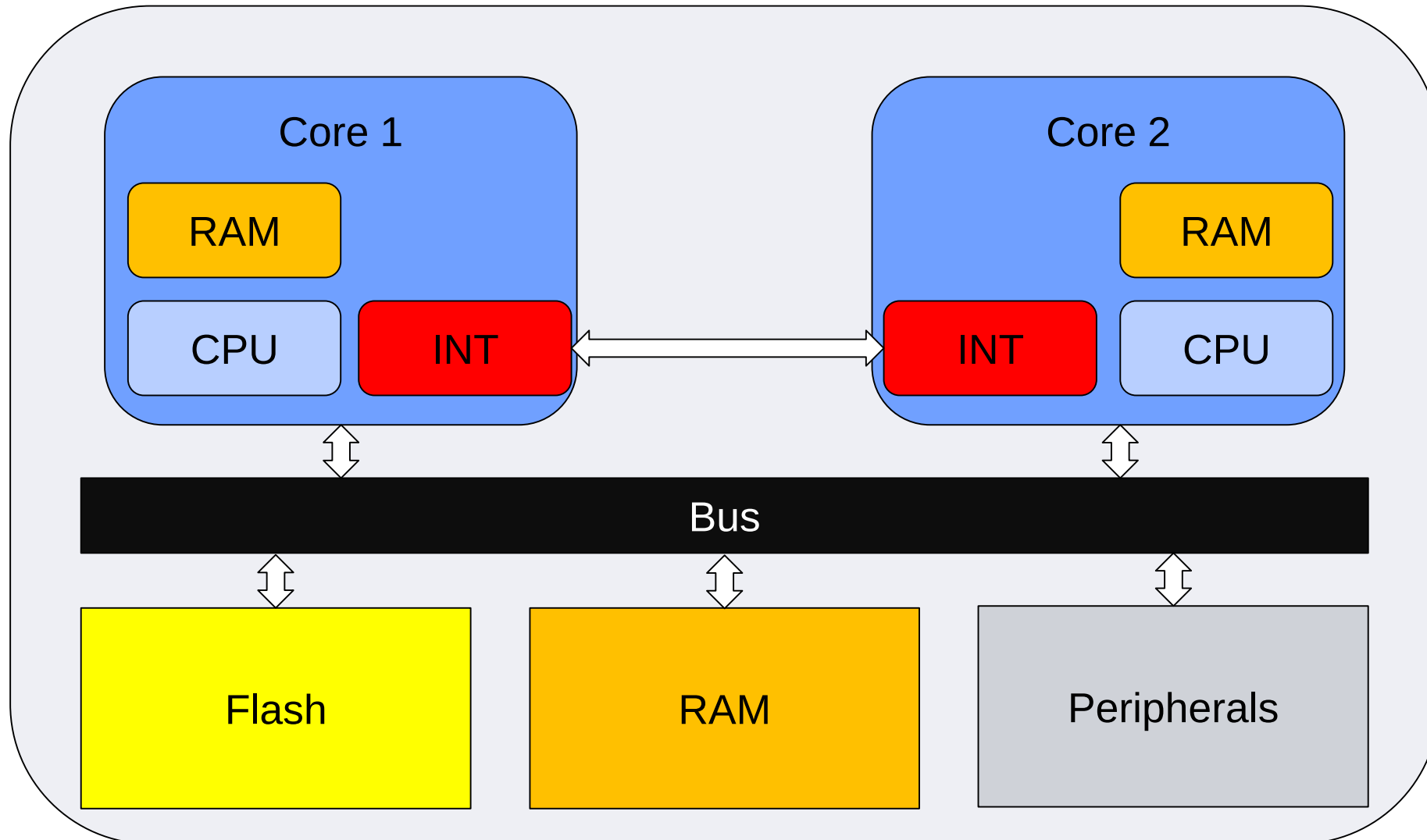


Heterogeneous Multicore

- Provide different cores for different domains
- Goal: Provide solutions for different requirement domains
- Example: ARM + NVIDIA GPGPU



Hardware Architectures - 2



Multi Threading, Multi Core and Multiple Discrete Core's

Multithreading

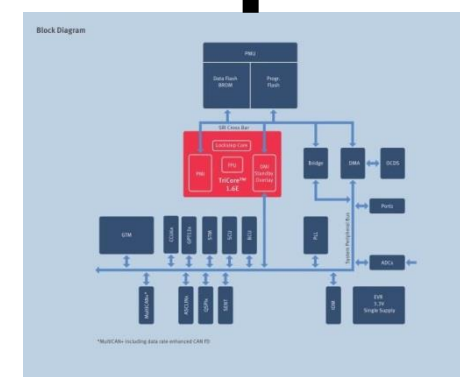
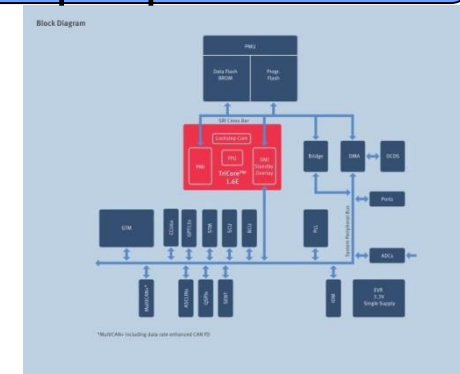
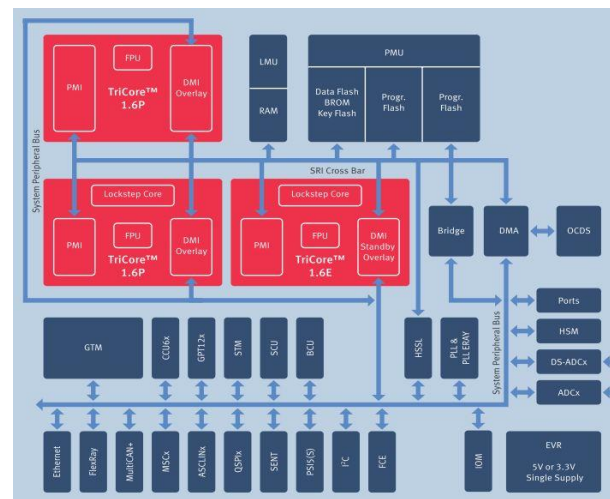
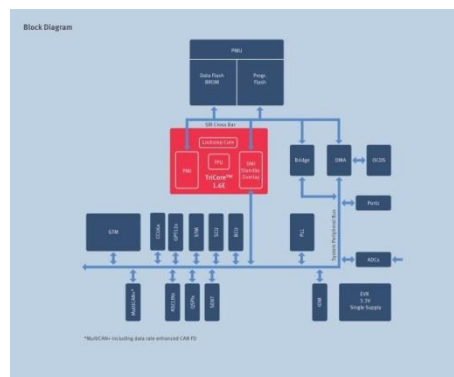
Real parallel execution of the code

Common memory, protected by MPU

Separated memory

Common peripherals, protected by MPU

Separated peripherals



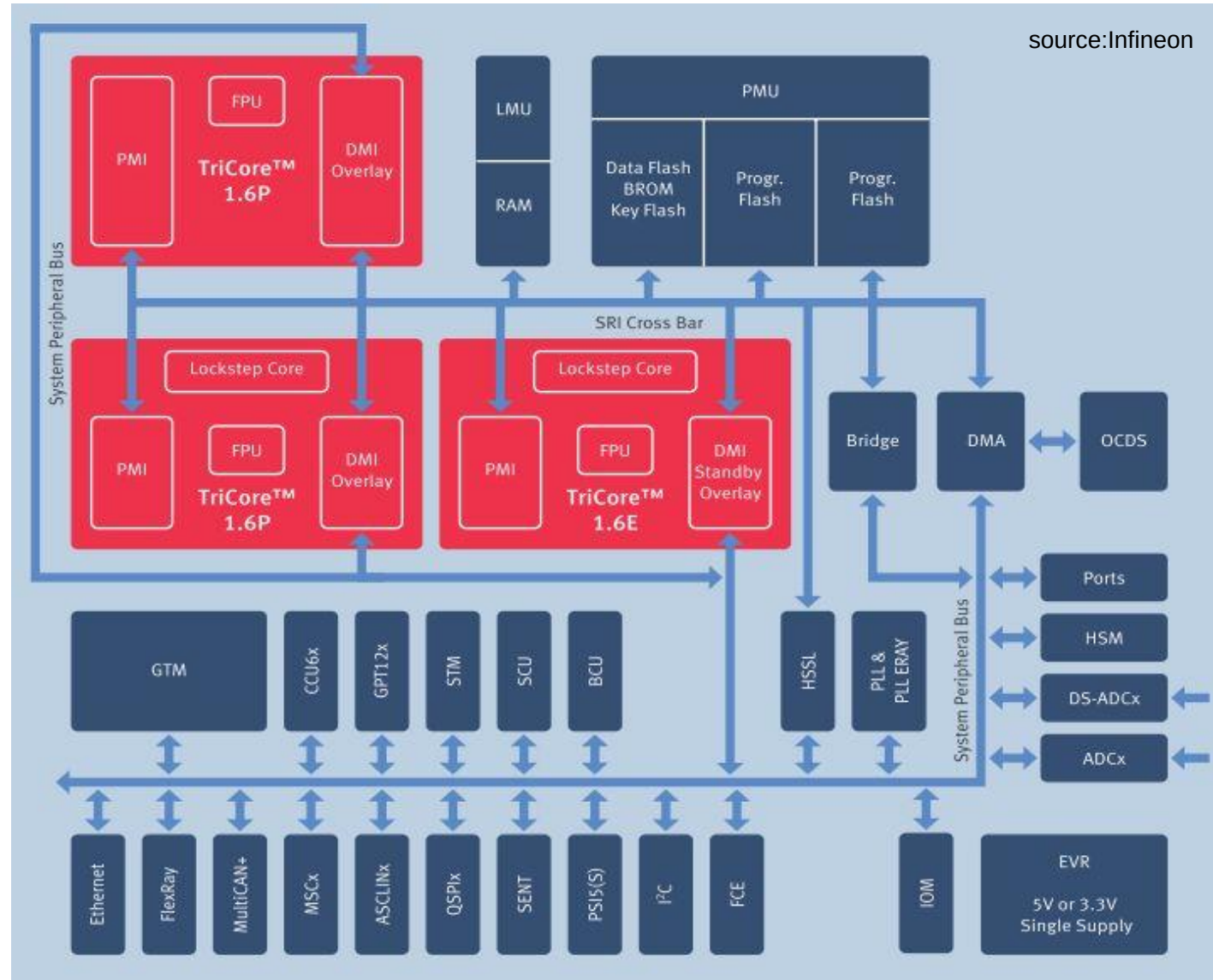
Infineon Aurix Platform – Generation 1

	TQFP-80	TQFP-100	LQFP-144 TQFP-144	LQFP-176	LFBGA-292	LBGA-416	LFBGA-516
9x Series up to 8MB Max. SRAM 2.75MB Triple-Core					TC297	TC298	TC299
7x Series up to 4MB Max. SRAM 472KB Triple-Core				TC275	TC277		
6x Series up to 2.5MB Max. SRAM 752KB Dual-Core			TC264	TC265	TC267		
4x Series up to 2MB Max. SRAM 96KB Single-Core			TC244				
3x Series up to 2MB Max. SRAM 452KB Lockstep-Core		TC233	TC234		TC237		
2x Series up to 1MB Max. SRAM 96KB Lockstep-Core	TC222	TC223	TC224				
1x Series up to 512KB Max. SRAM 56KB Lockstep-Core	TC212	TC213	TC214				

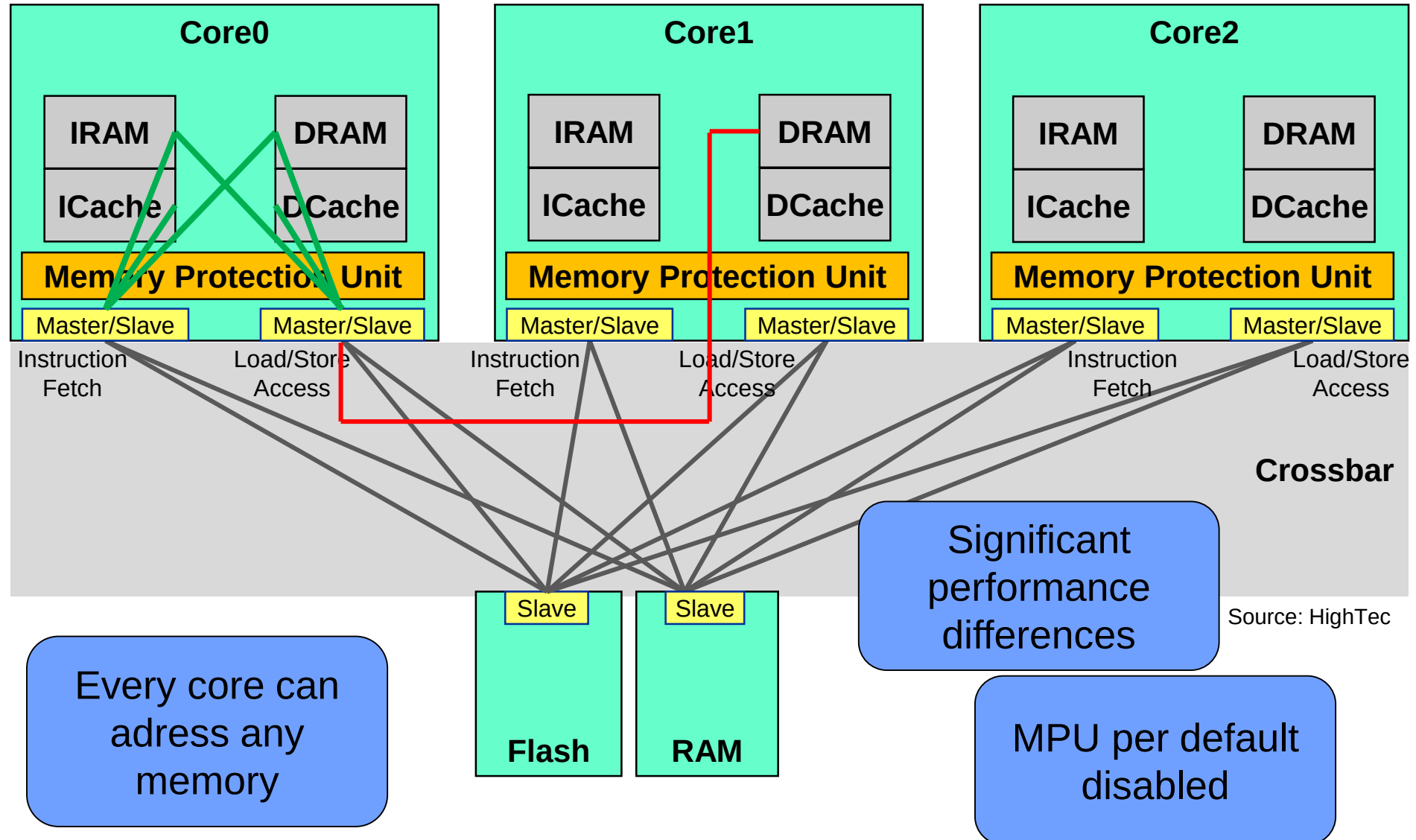
 Upgrade/Downgrade with pin-compatible packages

source:Infineon

Infineon Aurix TC277



Aurix Memory Access

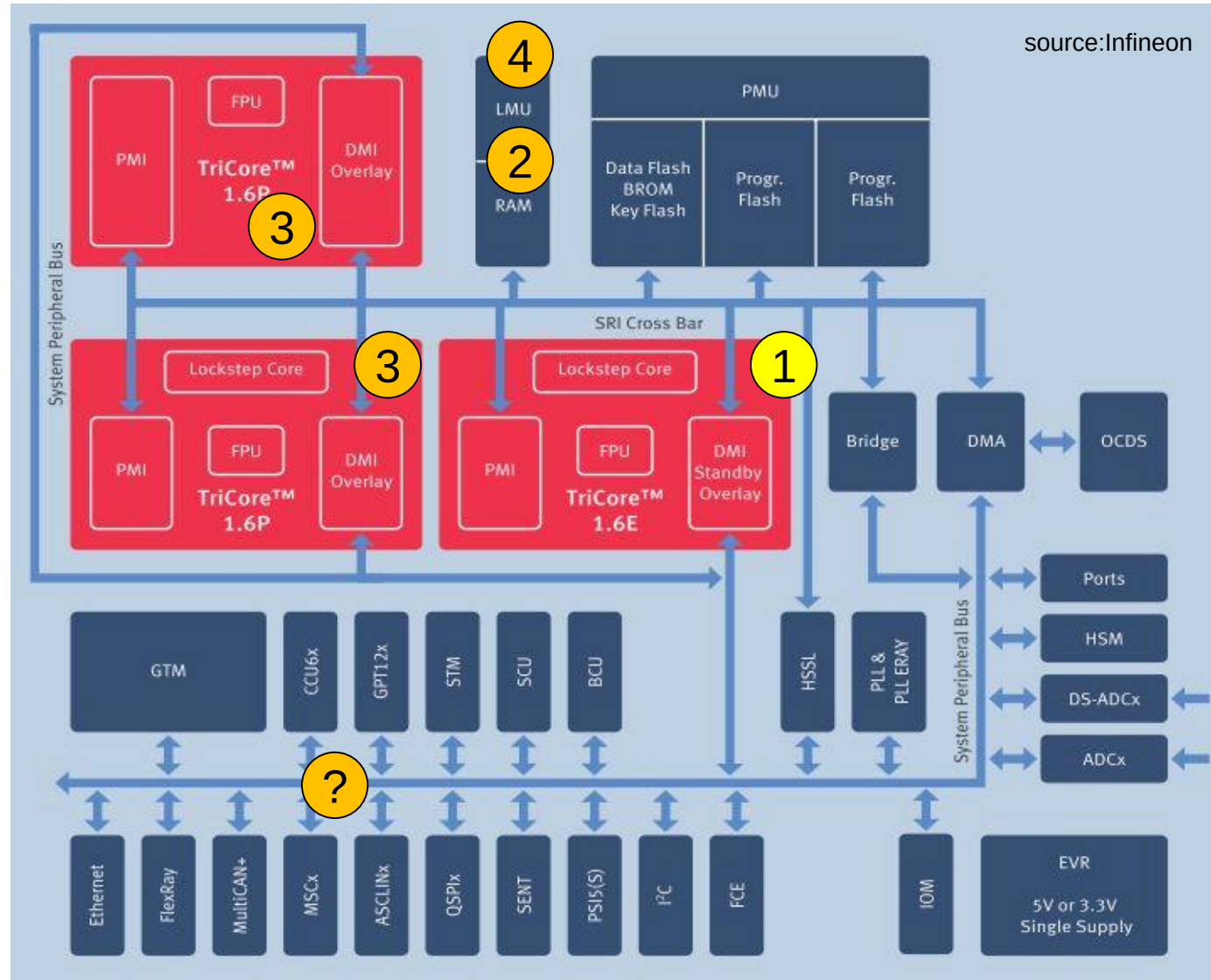


Aurix 2G

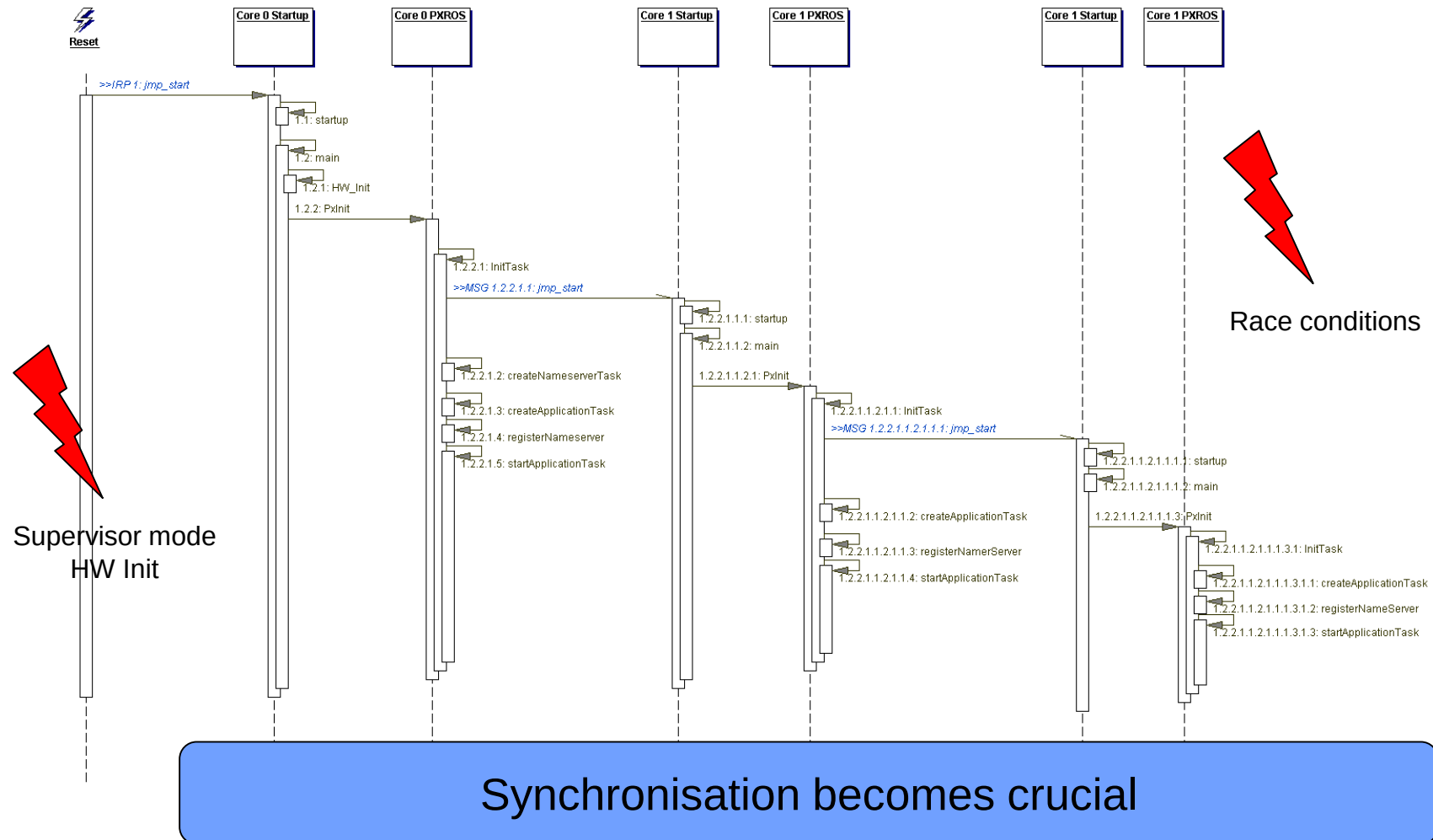
Feature Set		9x Series eXtension (16MB)	8x Series (10MB)	7x Series eXtended (3MB)	7x Series (6MB)	6x Series (4MB)	5x Series (4MB)	3x Series +eXtension (2MB)	3x Series (2MB)	2x Series (1MB)
TriCore 1.6	# Cores / Checker	6/4	4/2	3/3*	3/2	2/2	3/2	2/1	1/1	1/1
	Frequency	300MHz	300MHz	300MHz	300MHz	300MHz	300MHz	200MHz	200MHz	160MHz
Accelerator	Signal processing Unit (SPU)	2xSPU					2xSPU	1xSPU		
Flash	Program Flash	16MB	10MB	9MB	6MB	4MB	4MB	2MB	2MB	1MB
	Data Flash (physical/logical)	1024kB	512kB	256kB	256kB	128kB	128kB	128kB	128kB	96kB
SRAM	Total (DMI , PMI, LMU, AMU)	6912kB	1568kB	4208kB	1136kB	672kB	2837kB	1328kB	248kB	152kB
DMA	Channels	128	128	128	128	64	64	16	16	16
ADC	Modules Primary / Sec / FC / DS	8/4/8/14	8/4/4/10	4/4/2/6	4/4/2/6	4/2/2/4	2/0/0/0	4/2/0/0	2/2/0/0	2/2/0/0
	Channels Primary / Sec / FC / DS	64/64/8/14	64/64/4/10	32/64/2/6	32/64/2/6	32/32/2/4	16/0/0/0	>16/32/0/0	16/32/0/0	16/32/0/0
Timer	GTM TIM / (A)TOM / MCS	64 / 192 / 10	56 / 152 / 7	40 / 88 / 5	40 / 88 / 5	24 / 64 / 3	-	16 / 32 / 0	16 / 32 / 0	16 / 32 / 0
	CCU / GPT modules / bit streaming	2/1/1	2/1/0	2/1/0	2/1/-	2/1/0	2/1/1	2/1/1	2/1/0	2/1/0
Interfaces	FlexRay (#/ch.)	2 / 4	2/4	1/2	1/2	1/2	1/2	1/2	1/2	0/0
	CAN-FD / TT	12/1	12/1	12*/1	8/1	8/1	6/0	8/0	8/0	6/0
	QSPI / ASCLIN / I2C	6 / 12/2	5 / 24/2	5/12/1	5/12/1	4/12/1	4/4/0	4/12/0	4/12/0	4/6/0
	SENT / PSI5 / PSI5S	25/4/1	25/4/1	15/2/1	15/2/1	10/2/1	0/0/0	6/0/0	6/0/0	6/0/0
	HSSL / MSC / EBU	2/4/1	1/3/0	1/2/0	1/2/0	1/1/0	0/0/0	0/0/0	0/0/0	0/0/0
	Ethernet 100Mbps/1Gbps	1/1	1/1	1/1	1/1	1/1	1/1	1/1	-/-	-/-
	eMMC/SDIO	1/1		1/1				1/1		
	Radar /ext. ADC IF (RIF)	12x400Mbps LVDS	-	-	-	-	12x400Mbps LVDS	6x100Mbps LVDS	-	-
	Camera IF (CIF)	-	-	1	-	-	-	-	-	-
Security	HSM	HSM+ECC256	HSM+ECC256	HSM+ECC256	HSM+ECC256	HSM+ECC256	HSM+ECC256	HSM+ECC256	HSM+ECC256	HSM+ECC256
Safety	SIL Level	ASIL D	ASIL D	ASIL D	ASIL D	ASIL D	ASIL D	ASIL D	ASIL D	ASIL D
Power	EVR	Yes (3.3V/5V)	Yes (3.3V/5V)	Yes (3.3V/5V)	Yes (3.3V/5V)	Yes (3.3V/5V)	Yes (3.3V/5V)	Yes (3.3V/5V)	Yes (3.3V/5V)	Yes (3.3V/5V)
	Standby Control Unit	yes	yes	yes	yes	yes	yes	yes	yes	yes

source:Infineon

Aurix Startup - in a Nutshell



Aurix Startup – en Detail



Example Startup Code

```
void Start_Core(unsigned int coreId)
{
    if (coreId == 1)
    {
        /* enable CPU1 */
        CPU1_PC.U = (unsigned int)_start; /* set start address */
        CPU1_DBGSR.U = 0x4;                /* reset HALT state */
    }
    else if (coreId == 2)
    {
        /* enable CPU2 */
        CPU2_PC.U = (unsigned int)_start; /* set start address */
        CPU2_DBGSR.U = 0x4;                /* reset HALT state */
    }
}
```

Problem: These registers are also used by the debugger....

Which core am I – The CORE_ID Register

```
//Main is shared
int main(void)
{
    //Call some code on core 0
    if (__mfcr(CORE_ID) == 0) c0_loop();

    //Call some code on core 1
    if (__mfcr(CORE_ID) == 1) c1_loop();

    while(1);
}
```

A first hello world project...

- Let's try to start every core
- and to start func1(), func2() and func3() depending on the core

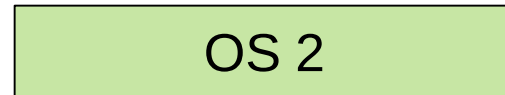


Multicore OS

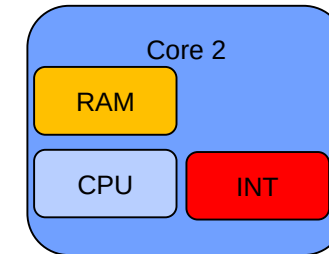
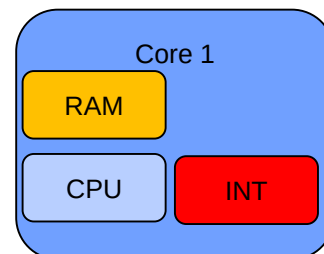
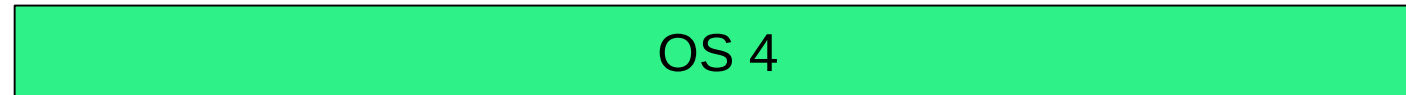
Variant 1



Variant 2



Variant 3



Additional requirements

Per core - All the known features like

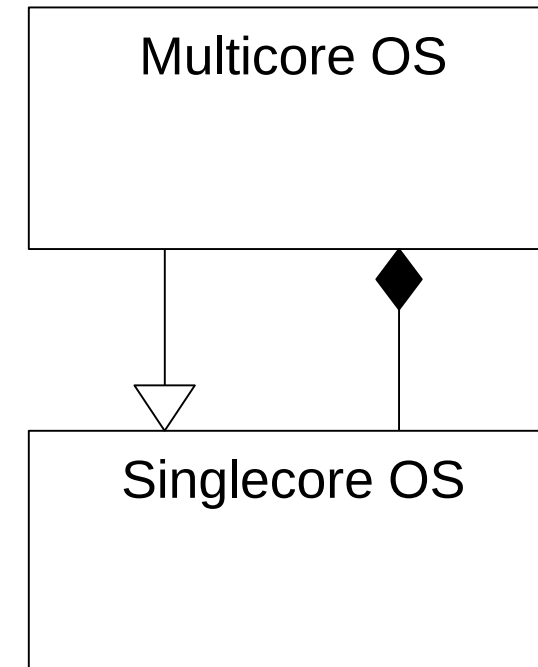
- Multithreading
- Intertask communication
- Events
- Semaphores

Mandatory extension

- Intercore communication
- Intercore synchronisation (Spinlocks)
- MPU support

Optional extension

- Load balancing



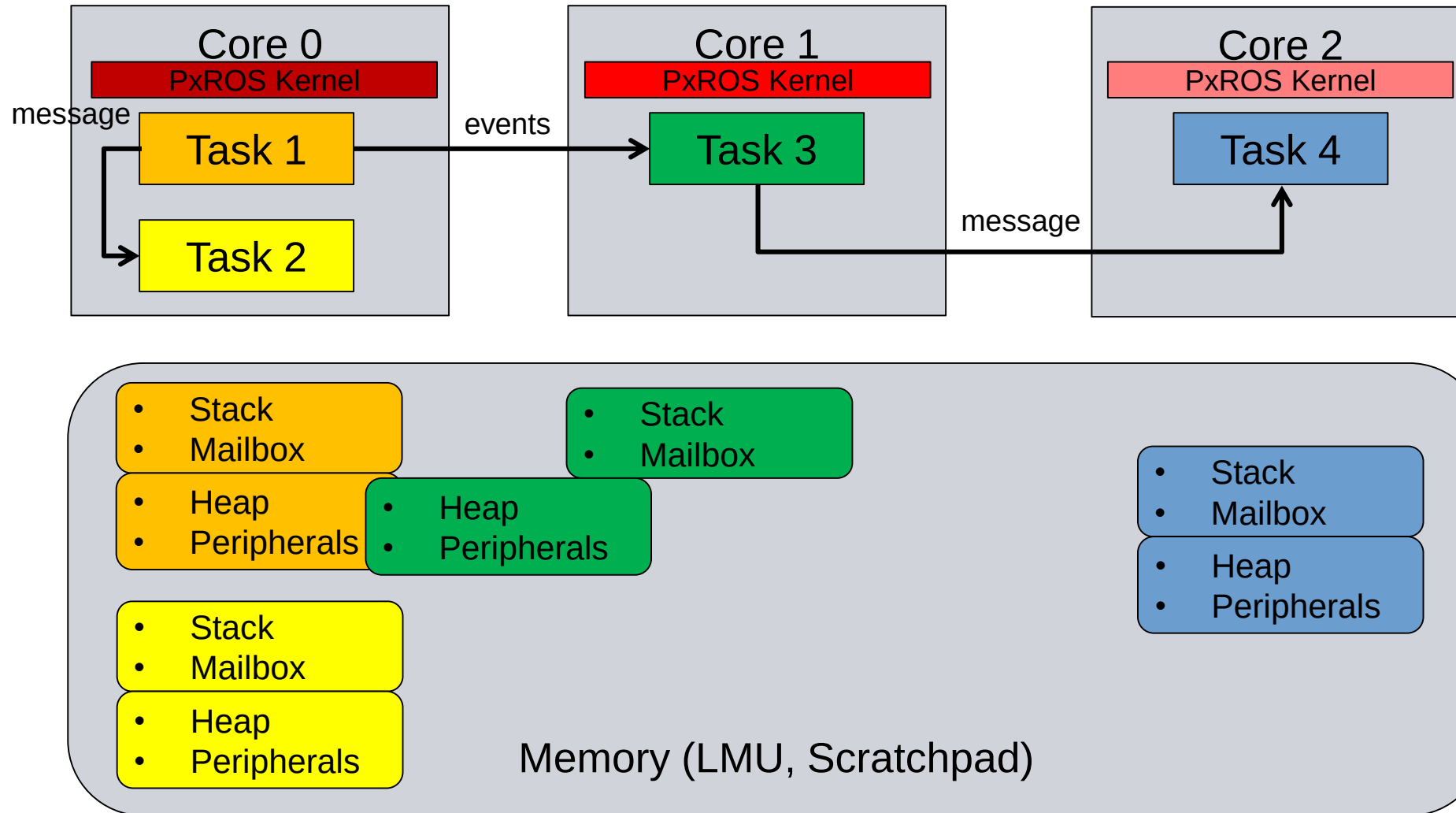
PXROS

- Embedded Operating System developed by HighTec (Saarbrücken)
- Optimized for Infineon Tricore
- Dynamic Operating System
- Comparable features like AutosarOS
 - Threads
 - Events
 - Messages
- MPU support
 - Trusted regions
 - Dynamic reconfiguration for message transfer
- Services
 - Nameserver
 - TCP/IP Stack
 - Trace

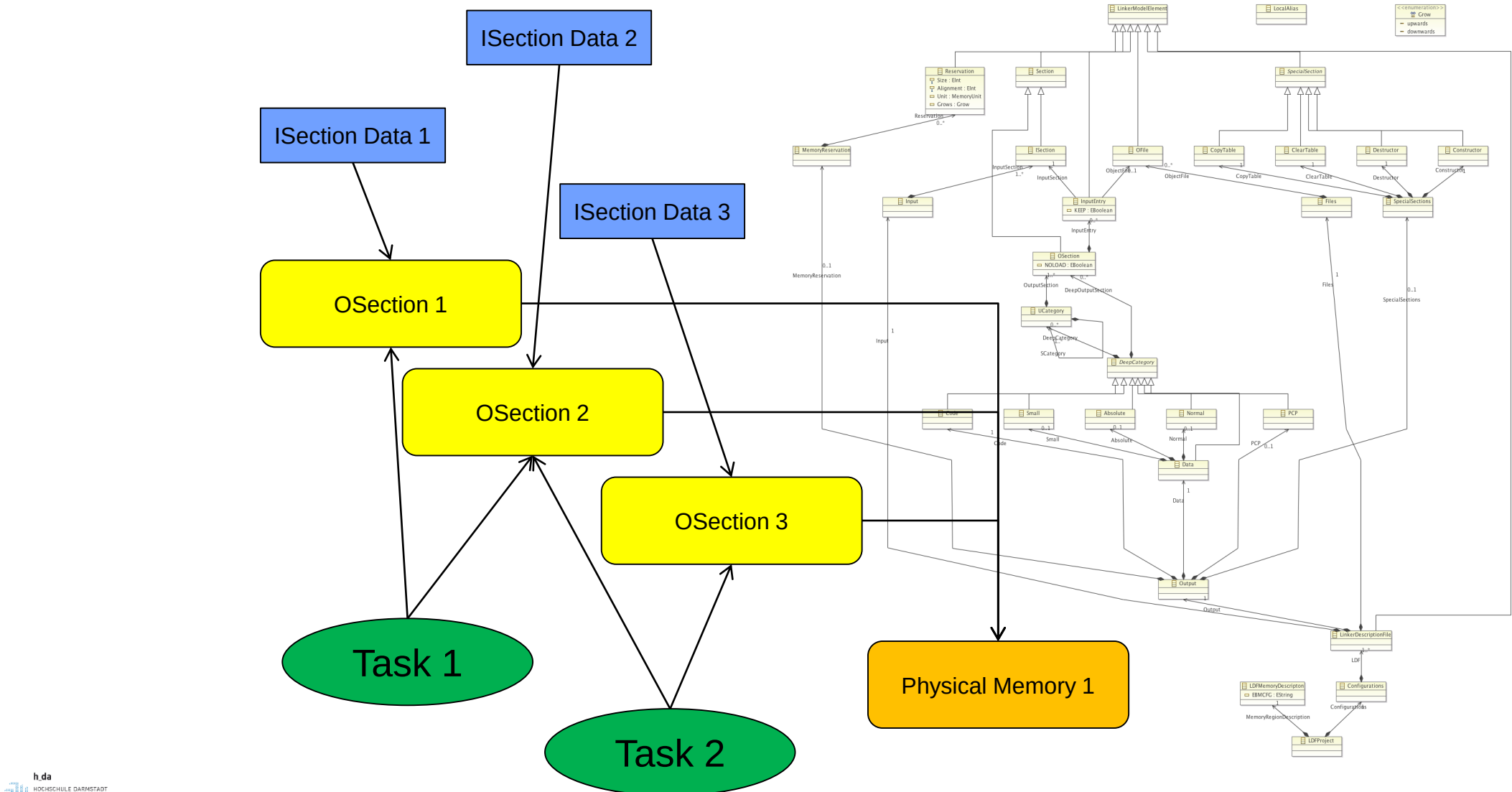


PXROS-HR
embedded safety

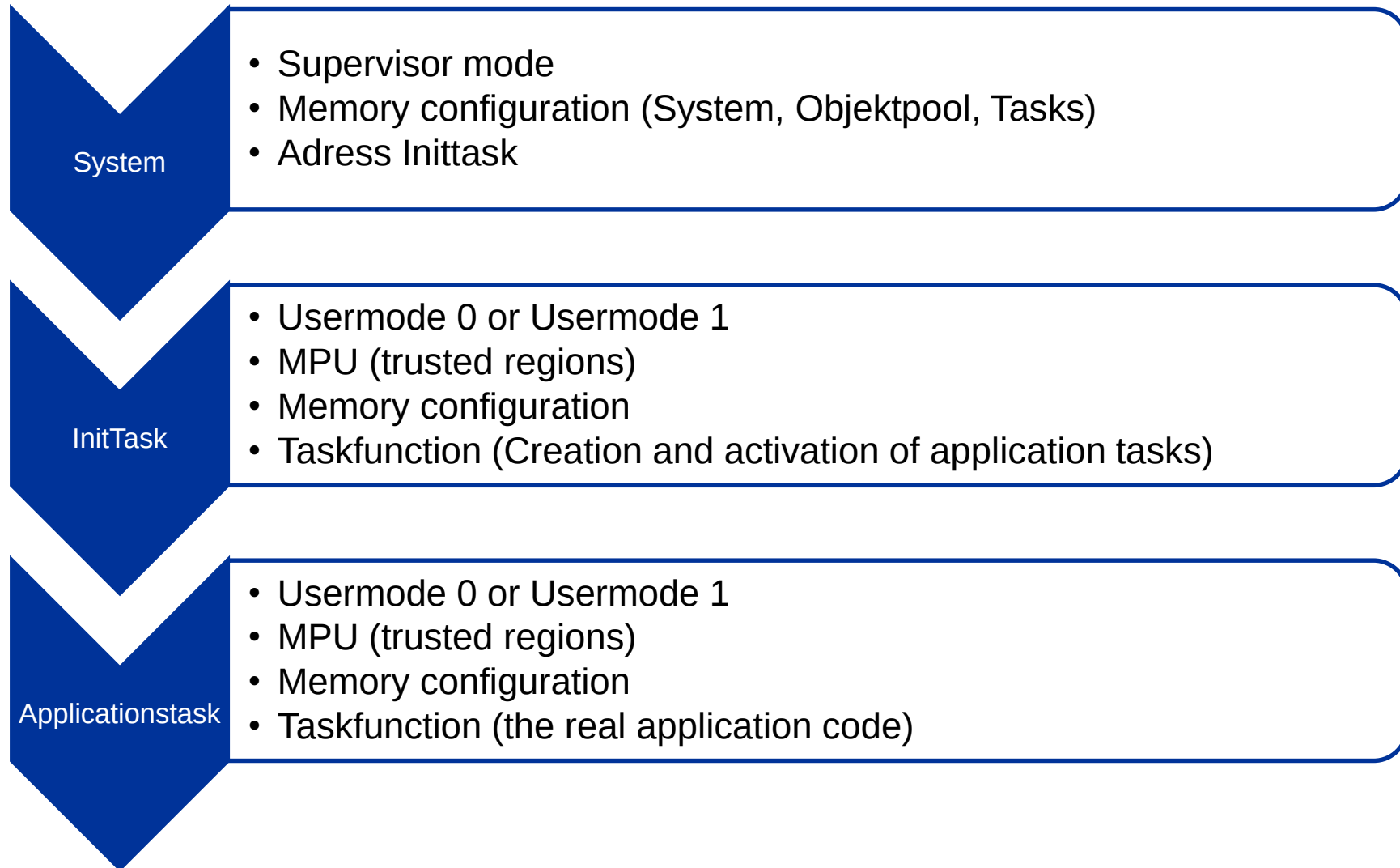
System Architecture with PXROS



Memory model with PXROS



PXROS Configuration

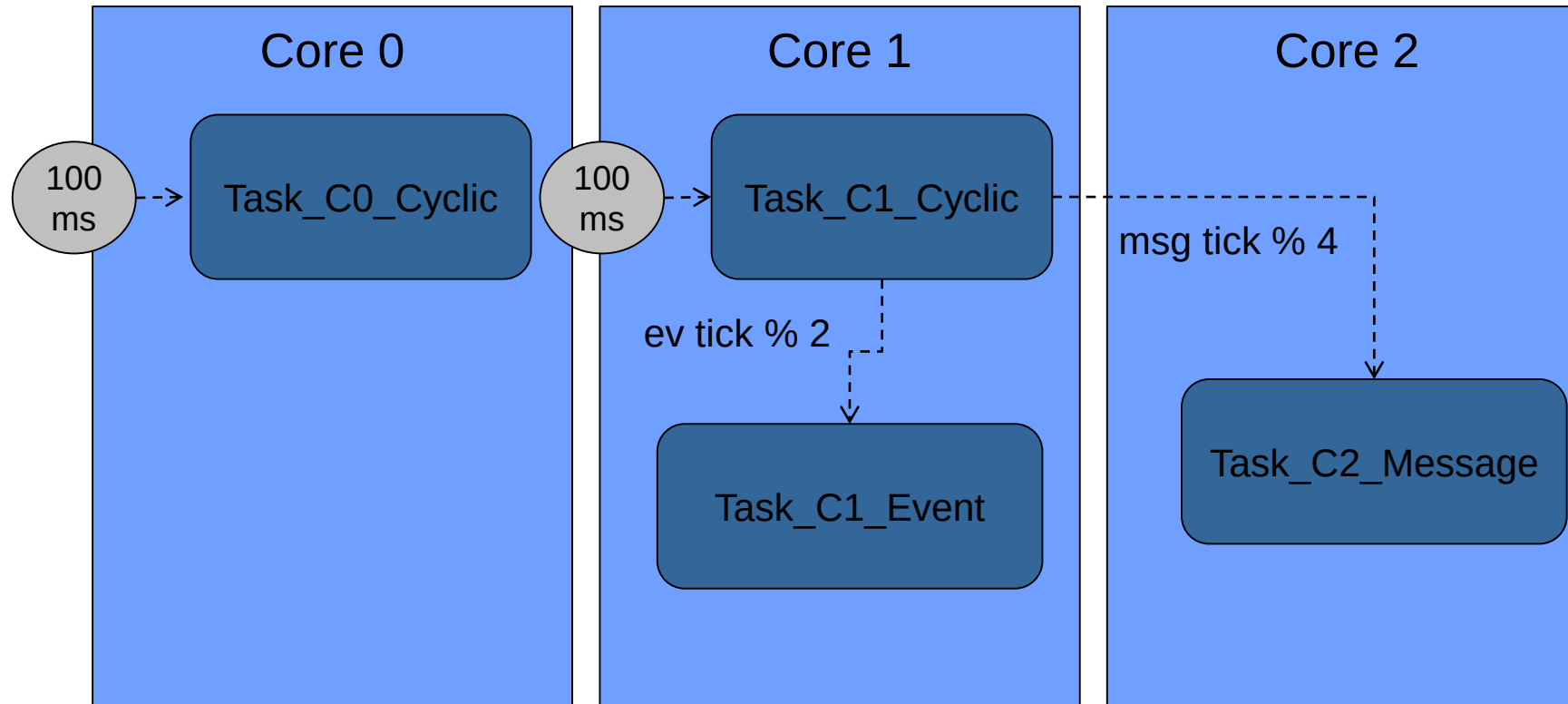


A first PXROS hello world project...

The blinking LED – one per core...



Architecture



Code Snippet - PXROS Configuration

System
memory
class

Object
memory
class

Task
memory
class

```
/* the PXROS System specification */
static const PxInitSpec_T InitSpec[CORE_COUNT] =
{
    {
        .is_sysmc_type      = PXMVarsizedAligned, // Manage memory in blocks of variable size.
                                                // Better memory utilization, slower than fixed blocks
        .is_sysmc_size      = 8,                // It is ignored in case Varsized system memory class
        .is_sysmc_blk       = Systemem, // Pointer to the system memory class, each core has each own memory space
        .is_sysmc_blksize   = SYSMEMSIZE_CORE0, // size of memory class

        .is_obj_number      = CORE0_NUM_OF_PXOBJS, // number of available PXROS-HR objects
        .is_obj_namelenhth  = PXROS_NAMESIZE,      // max string length for names
        .is_inittask        = &InitTaskSpec[0],    // Pointer to the specification of the init Task for Core_0
        .is_monitoring      = PXMonitorMemory,     // - ignored
        .is_schedext        = 0,                   // - ignored

        .is_objmc_type      = PXMVarsizedAligned, // Manage memory in blocks of variable size.
                                                // Better memory utilization, slower than fixed blocks
        .is_objlmc_size     = 8,                // It is ignored in case Varsized object memory class
        .is_objmc_blk       = GlobalObjmem,      // Pointer to the global object memory
        .is_objmc_blksize   = (PxSize_t)GLOBALOBJMEMSIZE, // size of object memory

        .is_taskmc_type     = PXMVarsizedAdjusted, // Manage task memory class in blocks of variable size.
        .is_taskmc_size     = 8,
        .is_taskmc_blk      = Taskmem_Core0,     // Pointer to the task memory class
        .is_taskmc_blksize  = TASKMEMSIZE_CORE0, // Task memory class size
    }
}
```

Code Snippet – InitTask Configuration

```
static const PxTaskSpec_T InitTaskSpec[CORE_COUNT] =
{
    {
        .ts_name = (const PxChar_t *)"InitTask_CPU0", // name of the task (registered in the Nameserver)
        .ts_fun = InitTask_Func, // Function which will be call after task activation
        .ts_mc = PXMctaskdefault, // Memory class for task - default
        .ts_opool = PXOpoolSystemdefault, // object pool - default

        /*
        * Available privileges:
        * Usermode 0 - allows access to the internal memory area exclusively (only)
        * Usermode 1 - allows access to the internal memory area and to the periphery
        *
        * Systemmode - allows full access (only available to the operating system and certain interrupt Handlers)
        *
        * */

        .ts_privileges = PXUser1Privilege, // allows access to the internal memory area and to the periphery

        .ts_accessrights = INITTASK_PXACCESS, /*access right of the first task
        the first task will get all available rights*/
        .ts_context = &InitTaskContext_0, // structure with information about addresses of memory
        //designed for data, stack
        // plus information about protection
        .ts_protect_region = InitTaskRegions, // Structure with regions which can be additionally accessed by InitTask
        .ts_taskstack =
        {
            .stk_type = PXStackFall,
            .stk_size = PXStackDontCheck,
            .stk_src.stk = &inittask0_stack[INITTASK_STACKSIZE] // address to the stack
        },
        .ts_inttaskstack =
        {
            .stk_type = PXStackFall, // - "PXStackFall": by the address following the stack area and
            // (optionally) its size in integers.
            .stk_size = PXStackDontCheck, // If the size should not be specified, it may be given as "PXStackDontCheck"
            .stk_src.stk = &inittask0_intstack[INITTASK_INTSTACKSIZE] // address to the interrupt stack
        },
        .ts_abortstacksize = 0 // used only by tasks using PXROS abort mechanism
        //(memory space needed to store task state in case of unexpected abort)
        //size of the abortstack in number of abortframes
    },
},
```

Code Snippet – Init Task Function

```
void InitTask_Func_C0(PxTask_t myID, PxMbx_t myMailbox, PxEvents_t myActivationEvents)
{
    // Turn off warning
    (void) myID;
    (void) myMailbox;
    (void) myActivationEvents;

    //Local Variable to check return status of PxRos functions
    PxError_t result;

    //Start core after initialisation of core 0 is finished
    Start_Core(1);

    /* Start PXROS time base with HZ (defined in clock.h) ticks per second. */
    //Todo - must be done on every core? Or only on core 0?
    TicksInit(1000);

    PxUInt_t id = PxGetCoreId();

    /* Initialize the name server - only on core 0 */
    PxTask_t NameSrv;
    NameSrv = PxNamesrvInit(NAMESRV_PRIO, PxNameServerSpaceBase, (PxUInt_t)PxNameServerSpaceEnd-(PxUInt_t)PxNameServerSpaceBase);
    if ( (result = PxTaskIdError(NameSrv)) != PXERR_NOERROR)
    {
        DET_stop(0,CREATENAMESEVER, result);
        //PxPanic();
    }

    // Needed in case messages are exchanged over core boundaries
    PxTask_t Task_ReleaseServer_Id;
    Task_ReleaseServer_Id = Task_ReleaseServerCreate(id,TASK_RELEASESESERVER_PRIO, TASK_RELEASESESERVER_ACTIVATION_EVENTS);
    if ((result = PxTaskIdError(Task_ReleaseServer_Id)) != PXERR_NOERROR)
    {
        DET_stop(0,CREATERELEASESEVER, result);
        //PxPanic();
    }

    //Task_ReleaseService_Id must run for PxGetGlobalNameServerMbx
    PxTaskSetPrio(PxGetId(), 15);

    //PXROS Service Tasks on core 0 finished
    //PROBLEM: HERE, THE PROGRAMM CRASHES
    //Core0_Initialized = BASE_SYSTEM_INIT;
    SYNC_SetC0(BASE_SYSTEM_INIT);

    //Some strange magic code suggested by Piotr
    PxTcpipMbx = PxMbxRequestMbx(_PxTcpAccessReqMbxId);
}
```

Possible Race
Not allowed anymore –
OS now boots all cores

Code Snippet – Cyclic Applicationstask

```
void CTask_C0_Cyclic::Task_Func(PxTask_t myID, PxBbx_t myMailbox, PxEvents_t myActivationEvents)
{
    // Turn off warning
    (void) myID;
    (void) myMailbox;
    (void) myActivationEvents;

    uint32_t count = 0;
    uint32_t loop = 0;

    //Generate a periodic event handler
    PxBbx_t pe = PxBbxRequest(PXOpoolTaskdefault, PE_TIME, PE_EVENT_TIME);
    PxBbxStart(pe);

    LEDCTL_On(0);

    while (1)
    {
        //Call runnables
        //HACK: Check sizeof operator
        for (loop = 0; loop < sizeof(RTL_cyclicHighPrioFunction_0)/sizeof(RTL_cyclicRunnable_t); ++loop)
        //for (loop = 0; loop < 0; ++loop)
        {
            if ((RTL_cyclicHighPrioFunction_0[loop].runnable != 0) && (count % RTL_cyclicHighPrioFunction_0[loop].cycleTime) == 0)
            {
                //Call runnable
                RTL_cyclicHighPrioFunction_0[loop].runnable();
            }
        }

        count += 10; //Increment by cycletime
        count %= 100000;

        //Wait for next timer event to come
        PxBbxWaitEvents(PE_EVENT_TIME);
    }
}
```

Multicore Application Development

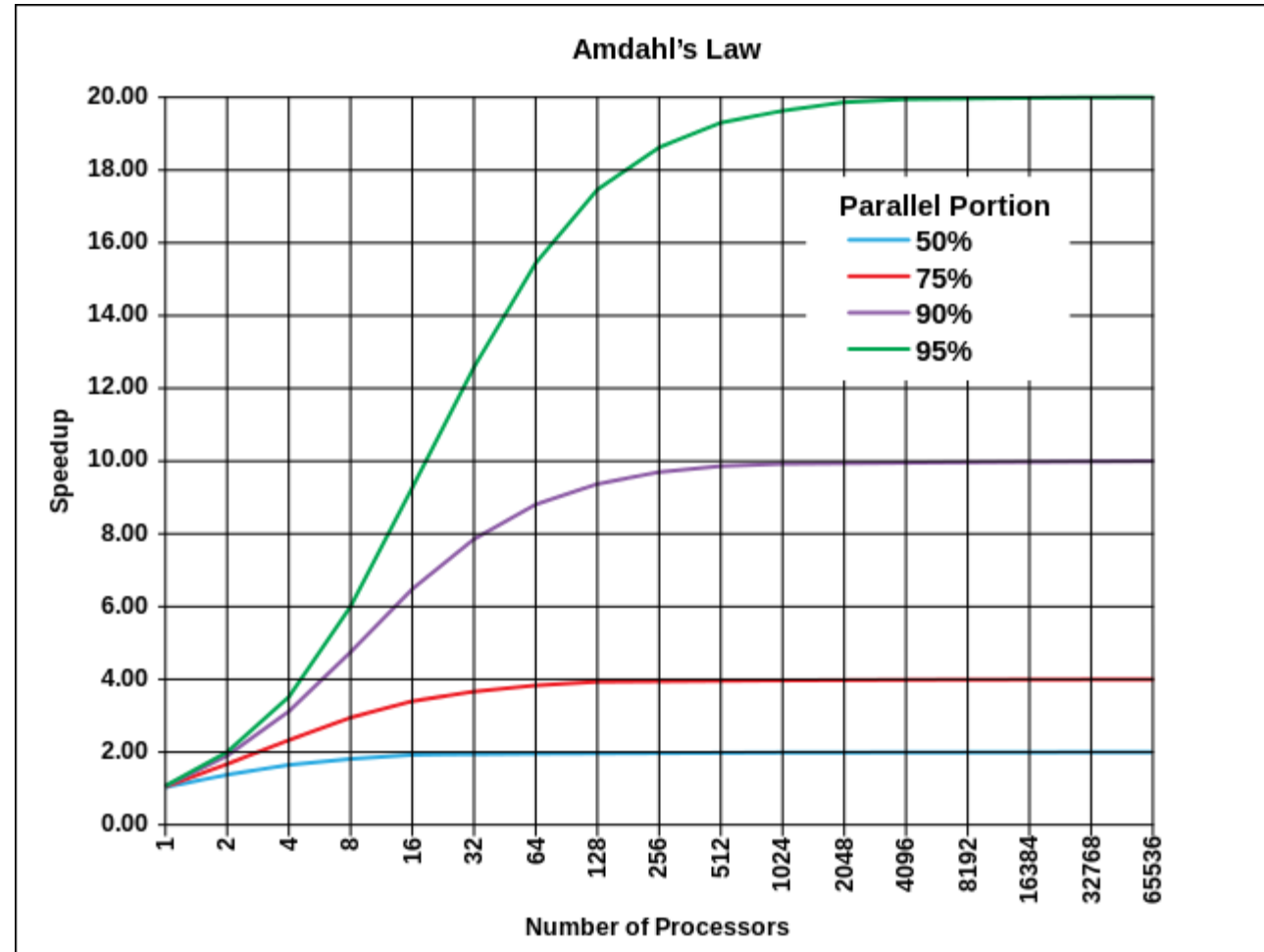
Some challenges....

- With increasing cores and memory, the functionality and complexity of the application will grow....
- Sequential coding can be complex, multithreading coding is complex, multicore coding is more than complex....
- CPU and memory partitioning becomes crucial...
- Debugging IS NO FUN!



Multicore Application Development

Performance – Amdahl's Law



source:wikipedia

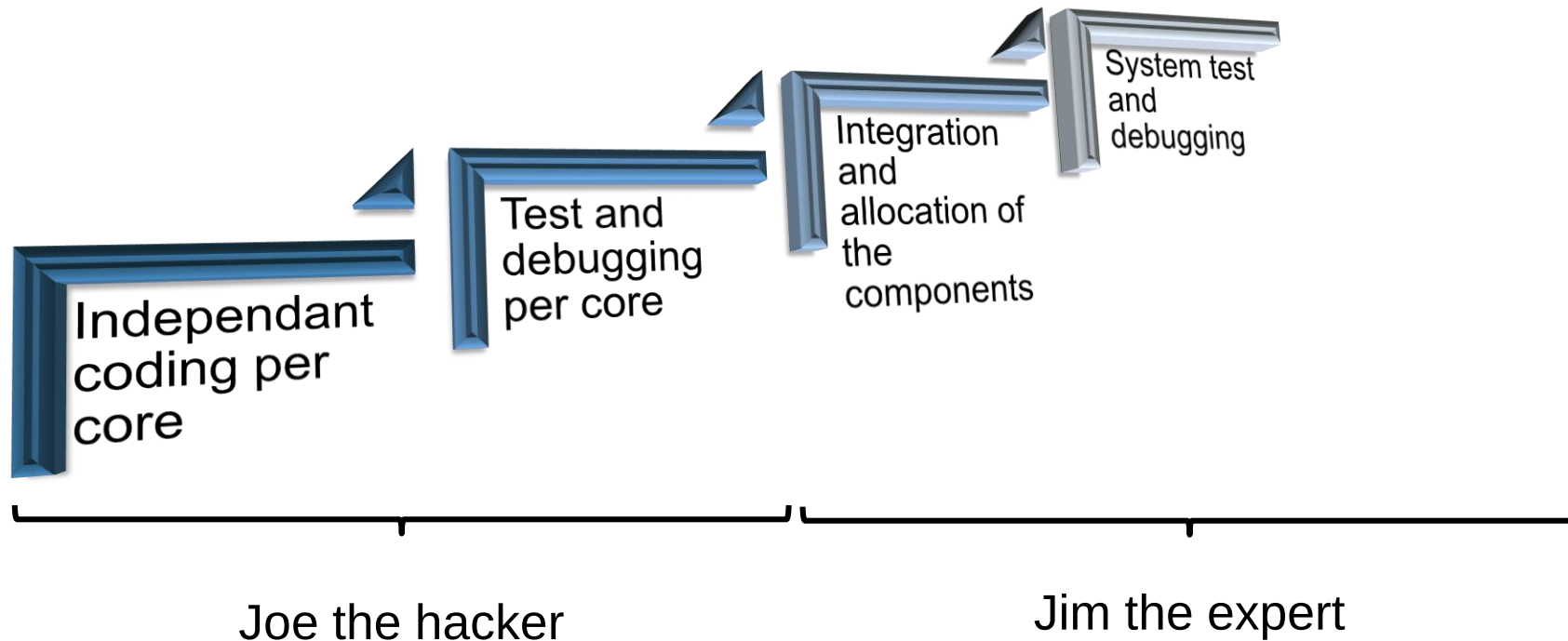
Key question – how can we take advantage of the multiple CPU?

- Try to implement the algorithm in a parallel fashion
 - Only few algorithms are suitable for this
 - Not supported by compilers
 - Some attempts have been made using model based development (e.g. Matlab)
- Distribute the code in encapsulated entities, which have no or only loose dependencies
 - Sounds like our runnable and RTE concept

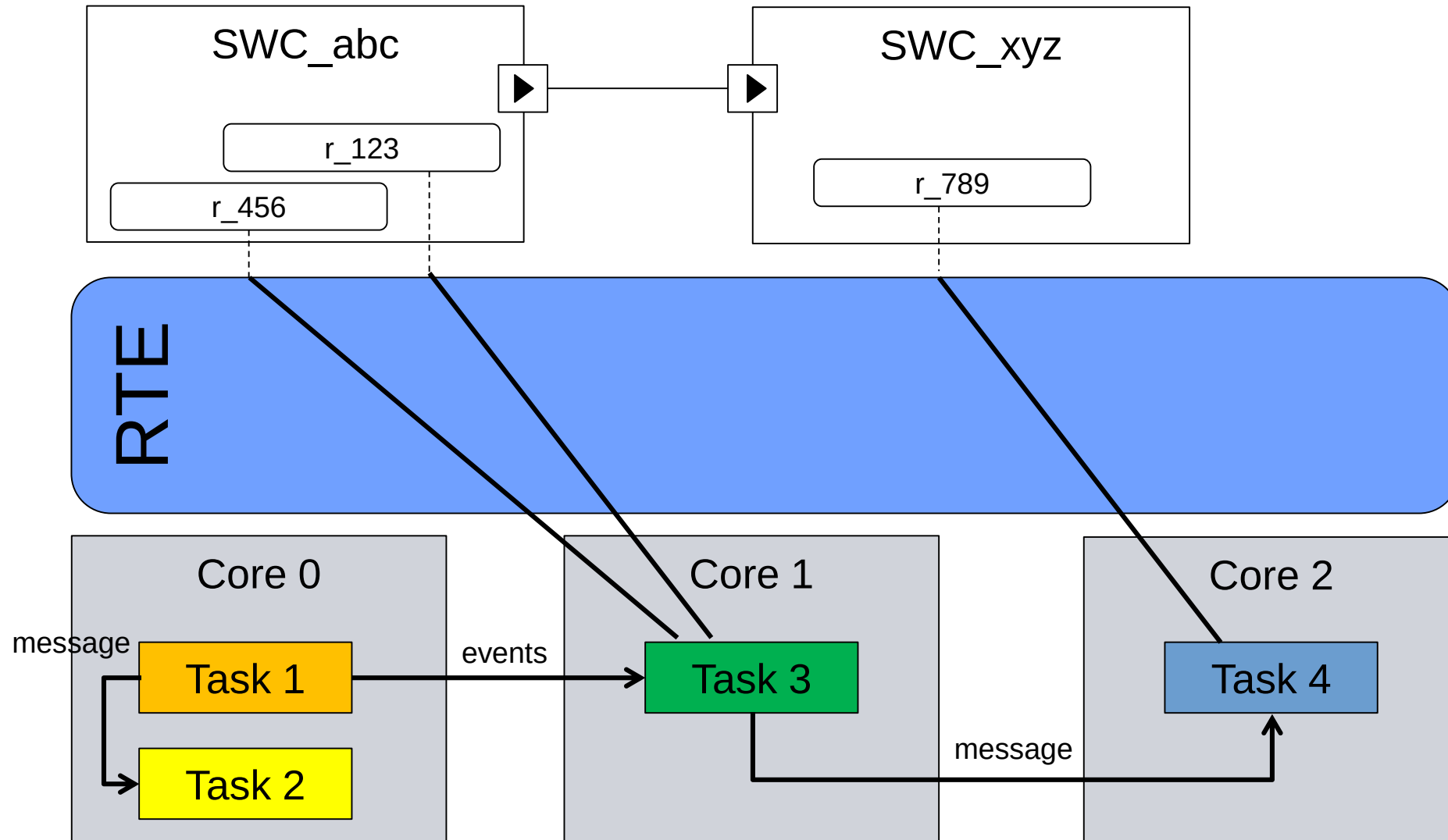
Multicore Application Development

General system design:

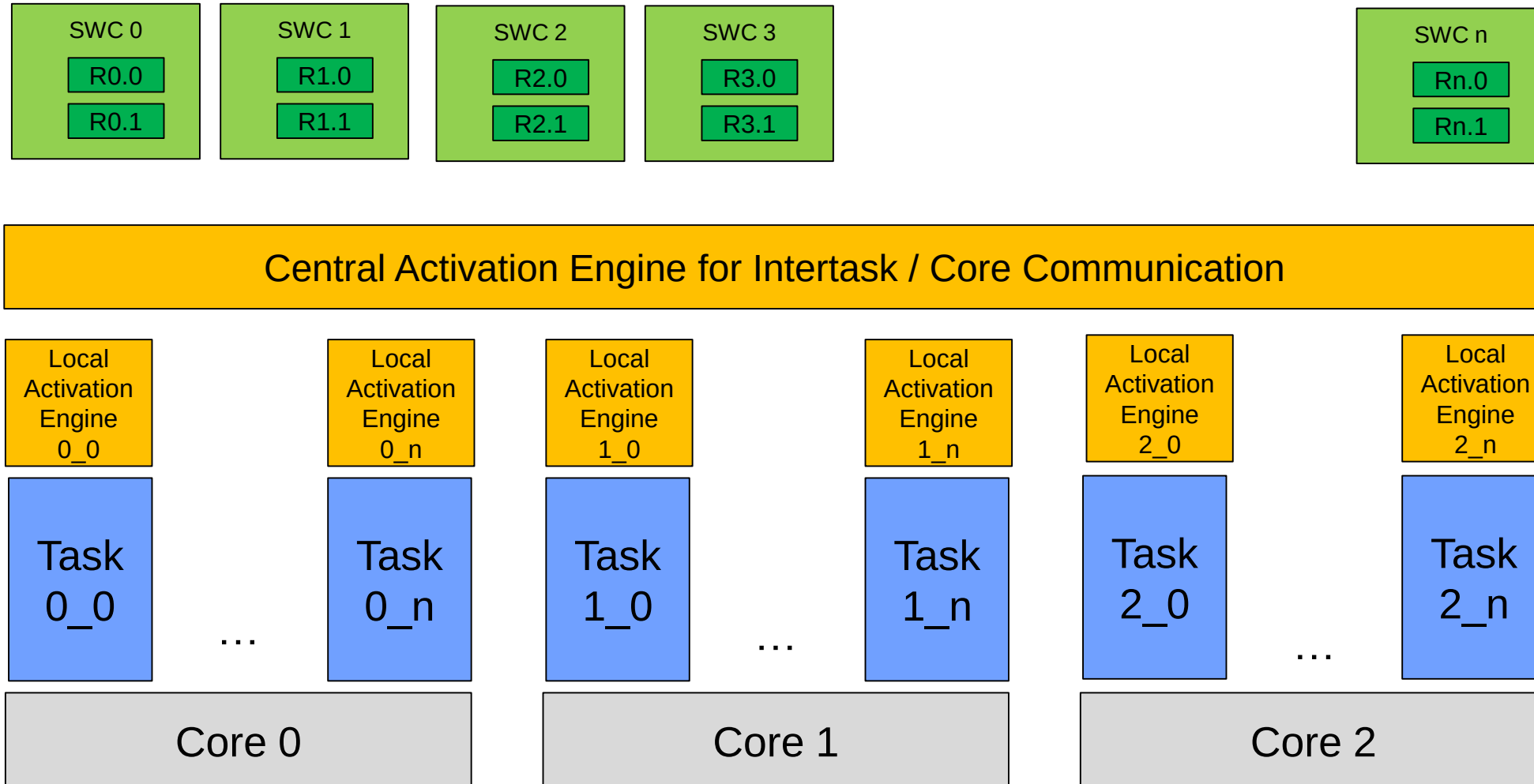
- loose coupling between the code on the different cores



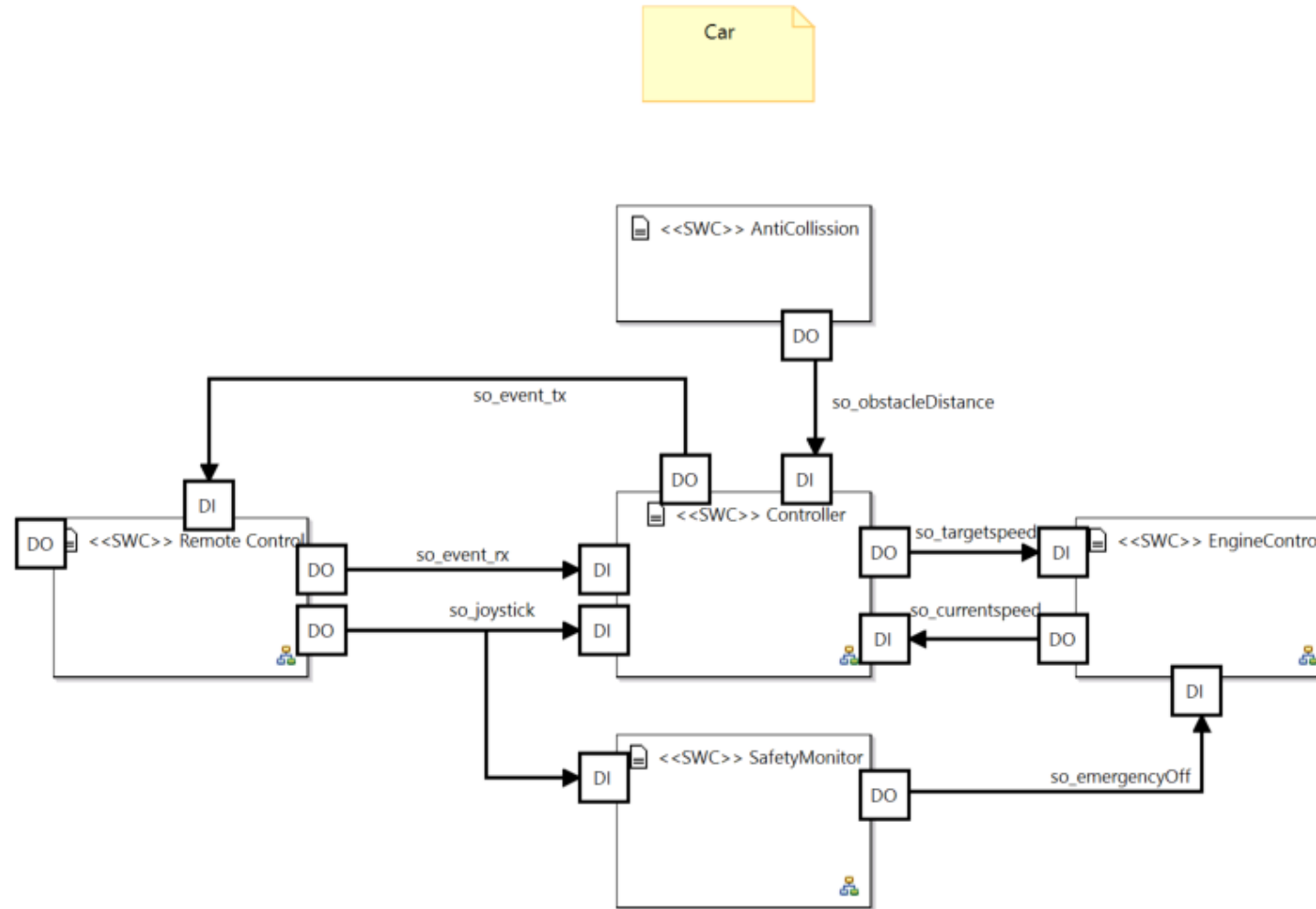
Multicore RTE



Technical Setup



Distribution on Cores and Tasks



Code Example – Task using a Local Activation Engine

```

/*****
 * Task Function
 *****/

void CTask_CO_Safety_Input::Task_Func(PxTask_t myID, PxMbx_t myMailbox, PxEvents_t myActivationEvents)
{
    // Turn off warning in case parameters are not used
    (void) myID;
    (void) myMailbox;
    (void) myActivationEvents;

    //create AE object
    AE_CO_Safety_Input AE_CO_Safety_Input;

    //===== Start normal task operation =====

    while (1){

        //wait for the next tick and execute runnables
        AE_CO_Safety_Input.WaitNextEv(0,0);
    }
}

```

The Triggers: Event and Message Reception

```
AE_runReturn_t AE_CCO_Safety_Input::WaitNextEv(const PxEvents_t additional_ev, PxMsgEvent_t* ev_triggered){  
  
    // result of invocation  
    AE_runReturn_t result=AE_RUNRETURN_OK;  
  
    //result of the invocation of cyclic runnables  
    AE_runReturn_t result_cyclic;  
  
    //event buffer  
    PxMsgEvent_t ev_buffer;  
  
    //Wait for next event to come  
    ev_buffer=PxMsgReceive_EvWait(this->m_mailbox, AE_CYCLICEV_MSK|additional_ev);  
  
    //check if the event was a timer tick  
    if(0!=(AE_CYCLICEV_MSK&ev_buffer.events)){  
        //invoke cyclic runnables  
        result_cyclic=this->InvokeCyclicRunnables();  
  
        //check result  
        if(AE_RUNRETURN_OK!=result_cyclic){  
            result=result_cyclic;  
        }  
    }  
  
    //safe event for user if container is provided  
    if(NULL!=ev_triggered){  
        *ev_triggered=ev_buffer;  
    }  
  
    // return result  
    return result;  
};
```

Explicit function calls instead of tables – better debug-ability / analyzability

```
INLINE AE_runReturn_t AE_CCO_Safety_Input::InvokeCyclicRunnables(void){

    // result of invocation
    AE_runReturn_t tableResult=AE_RUNRETURN_OK;

    //get current availability mode
    AE_Availability_t current_availability=AE_StateManager.getCurrentAvailability();

    //in case a runnable invocation with result is failing, the ID of the failed runnable is stored in this variable
    int tableIndex = -1;

    //invoke runnables for the current mode
    switch(current_availability){
    case AE_comfort: /* Lowest availability. Active during normal operation, including the whole feature set. */

        // ----- REMOTE_run_checkJoystickZero; SWC "Remote"; Order 0; Guards: SafetyInputInitialized; Triggers: 100ms
        if((true==SafetyInputInitialized())){
            if (TRUE == AE__calculateSafetyRequest(REMOTE_run_checkJoystickZero(), tableResult))
                tableIndex = 0;
        }

        // ----- REMOTE_run_checkJoystickAge; SWC "Remote"; Order 1; Guards: none; Triggers: 100ms
        if (TRUE == AE__calculateSafetyRequest(REMOTE_run_checkJoystickAge(), tableResult))
            tableIndex = 1;

        // ----- REMOTE_run_checkJoystickData; SWC "Remote"; Order 2; Guards: none; Triggers: 100ms
        if (TRUE == AE__calculateSafetyRequest(REMOTE_run_checkJoystickData(), tableResult))
            tableIndex = 2;

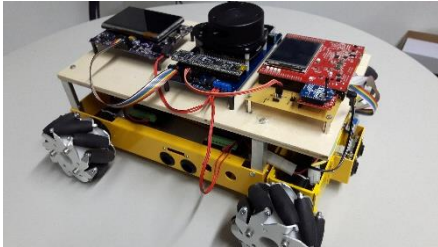
        // ----- LIDAR_run_checkCollissionSensor; SWC "LiDAR"; Order 3; Guards: none; Triggers: 100ms
        if (TRUE == AE__calculateSafetyRequest(LIDAR_run_checkCollissionSensor(), tableResult))
            tableIndex = 3;

        break; /* AE_comfort */
    default:
        break;
    }
}
```

Wrapup

- Multicore Hardware
- Higher complexity due to real parallel execution
- Memory configuration is crucial
- Concept of a multicore runtime environment

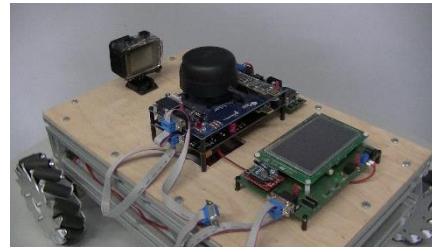
What we have achieved – 3 Cars but (almost) same codebase



Aurix Multicore

PXROS

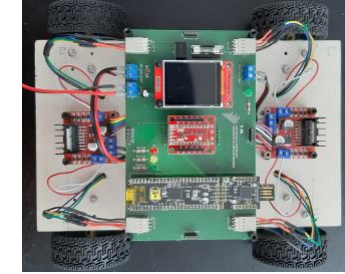
CanOpen
Distributed



STM F7 Single
Core

FreeRTOS

CanOpen
Distributed



PSOC Single
Core

Erika

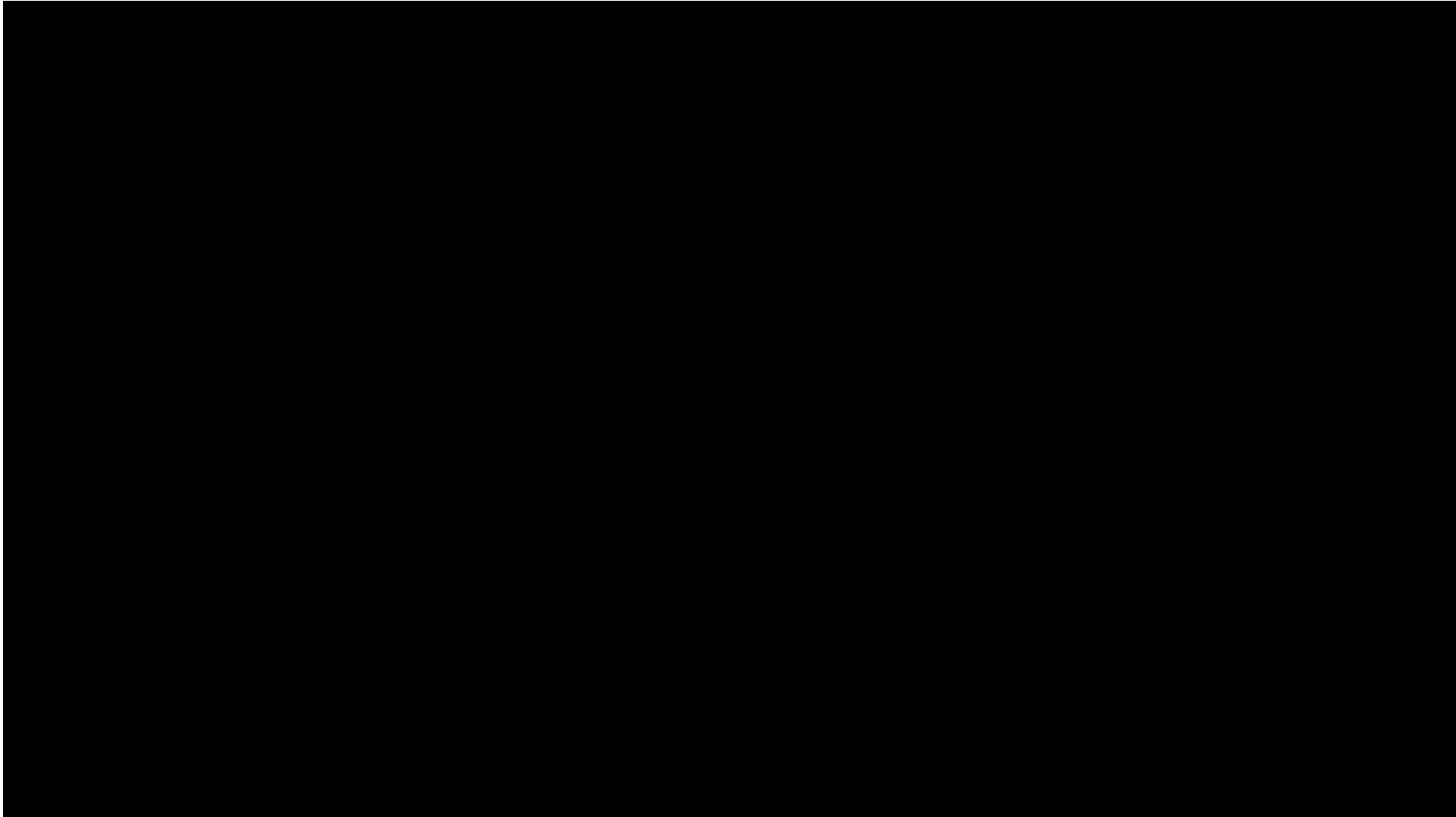
Direct MCAL

C++ Application Code

Minor
Modifications

C Application Code

CAR 5 - Mercanum



What you should take from this lecture

- When moving to complex embedded systems, we need a clear architecture (HW + SW)
 - Selection of HW components (performance parameters)
 - What data is processed where (data flow)
 - What logic is executed when (control flow)
- Although the AUTOSAR RTE concept is complex
 - We could easily add new parts (Remember: adding software is easy, changing software is hard)
 - And could track problems to the root cause
 - Especially (functional) failure monitoring was easily added
- There are a couple of services we need in every project → your own embedded library!
 - Logging
 - Diagnostics
 - Ringbuffers
 - Filters

Don't trust hardware to work as it should!